Microsoft Agent Framework Fundamentals

A Complete Guide

9 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/maf-fundamentals/

← Back to Tutorials

1. Foundations & First Steps

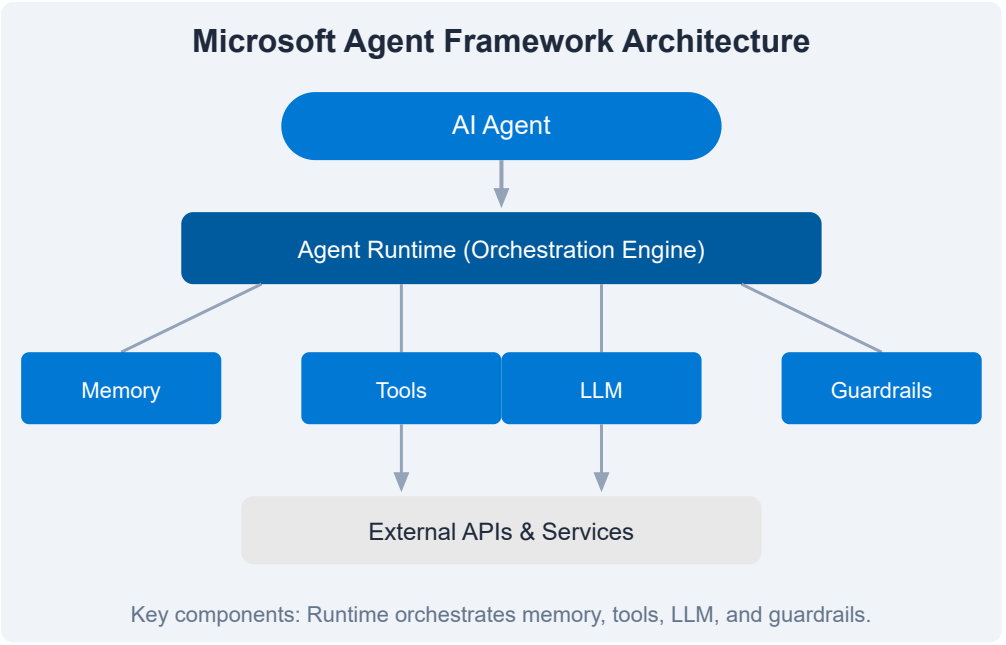
What is the Microsoft Agent Framework?

The Microsoft Agent Framework (MAF) is a set of SDKs, runtimes, and cloud services for building, deploying, and managing intelligent AI agents. Agents are autonomous or semi-autonomous programs that use large language models (LLMs) to reason, execute tools, maintain memory, and interact with users and systems.

Core Architecture

Every MAF application has four layers:

- **Agent** — the AI entity with instructions, memory, and tools
- **Orchestrator** — manages the agent lifecycle, turn-taking, and multi-agent handoffs
- **Runtime** — execution environment that coordinates LLM calls, tool execution, and state
- **Channels** — user-facing interfaces (Teams, web chat, custom apps)



Key components: Runtime orchestrates memory, tools, LLM, and guardrails.

Key Concepts

Concept	Description
Agent	An AI-powered entity that can reason, act, and respond
Memory	Persistent storage of conversation history and user context
Tool	Functions that agents can invoke (APIs, DB queries, calculators)
Orchestrator	Coordinates multiple agents and handoffs between them
Guardrails	Safety and compliance policies applied to agent inputs/outputs

Why MAF?

- Deep integration with Microsoft ecosystem (Azure, Teams, Copilot)
- Built-in support for memory, tools, and multi-agent orchestration
- Enterprise-grade governance, security, and observability
- Open SDK model — works with any LLM provider

💡 **Key Insight:** Agents differ from traditional chatbots by maintaining state, executing actions, and autonomously deciding which tools to call.

🖍️ **Exercise:** Write down three real-world problems you could solve with an AI agent. For each, list: (1) what data it would need, (2) what tools it would call, and (3) what memory it should preserve.

← Back to Tutorials

2. Environment Setup

Prerequisites

- **Node.js 18+** or **.NET 8+** SDK
- **Azure subscription** (free tier is sufficient)
- **Visual Studio Code** or any code editor
- **Git** for version control

Installing the MAF SDK

For JavaScript/TypeScript (npm)

```
mkdir my-agent
cd my-agent
npm init -y
npm install @microsoft/agents @azure/identity
```

For .NET

```
dotnet new console -n MyAgent
cd MyAgent
dotnet add package Microsoft.Agents
dotnet add package Azure.Identity
```

Project Structure

```
my-agent/
├── src/
│   ├── agent.ts          # Agent definition
│   ├── tools/            # Custom tool implementations
│   └── memory/           # Memory providers
├── config/
│   └── settings.json     # Agent configuration
├── package.json
└── .env                  # Environment variables
```

Course Setup & Lab Enrollment

Many MAF courses include interactive lab environments. To enroll:

1. Navigate to the lab portal provided by your instructor or course
2. Sign in with your Microsoft or GitHub account
3. Select the "Agent Framework Fundamentals" lab module
4. Follow the setup wizard to provision your sandbox

> 📘 **Note:** Lab environments include pre-installed SDKs, sample code, and an Azure sandbox. No credit card required for course labs.

Verifying Your Setup

```
# Node.js
node -e "const maf = require('@microsoft/agents'); console.log('MAF SDK loade

# .NET
dotnet list package | findstr Agents
```

> ✏️ **Exercise:** Create a new project using the MAF SDK. Initialize a Git repository, create the project structure above, and run the verification command to confirm the SDK loads without errors.

← Back to Tutorials

3. Build Your First Agent

Hello World Agent

Let's build a simple agent that greets users and remembers their name.

JavaScript / TypeScript

```
import { Agent, AgentClient } from "@microsoft/agents";
import { DefaultAzureCredential } from "@azure/identity";

const agent = new Agent({
  name: "GreeterAgent",
  instructions: "You are a friendly greeter. Greet the user warmly and rememb
  model: "gpt-4o",
  auth: new DefaultAzureCredential(),
});

const client = new AgentClient(agent);
const response = await client.sendMessage("Hi, I'm Snehal!");
console.log(response.text);
```

Agent Configuration

Property	Description	Example
`name`	Unique identifier	`"GreeterAgent"`
`instructions`	System prompt defining behavior	`"You are a friendly..."`
`model`	LLM to use	`"gpt-4o"`
`temperature`	Creativity (0-1)	`0.7`

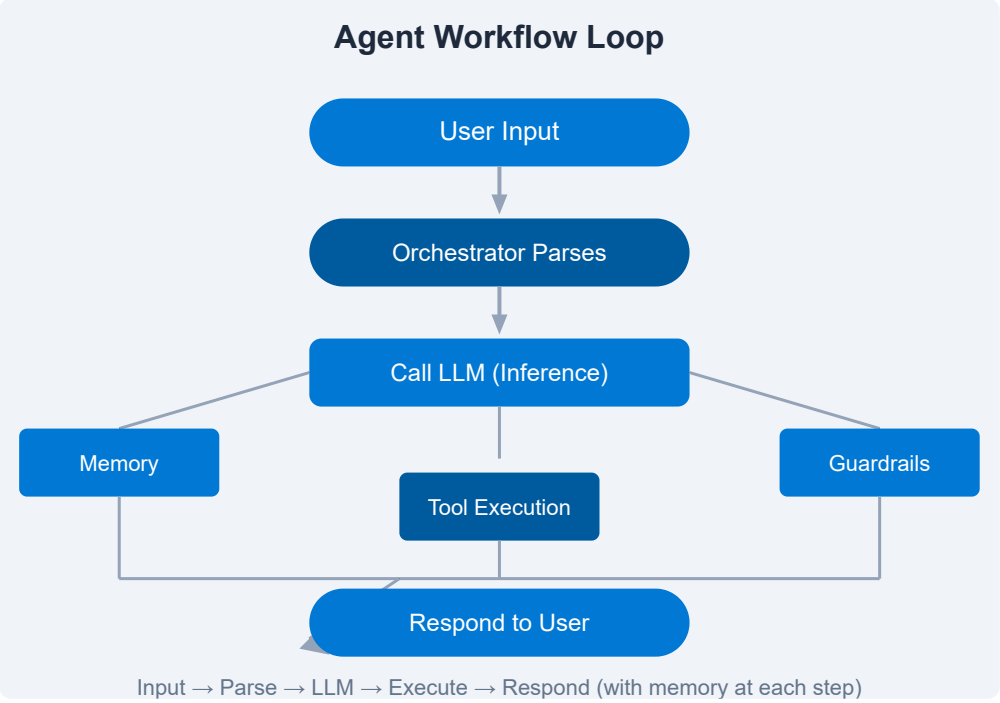
Prompt Design Best Practices

- Be specific about the agent's role and boundaries
- Include examples of expected interactions (few-shot)
- Define what the agent *should not* do
- Keep instructions concise but comprehensive

Agent Lifecycle

An agent progresses through these states:

1. **Created** — definition saved
2. **Configured** — memory, tools, and guardrails attached
3. **Active** — ready to receive messages
4. **Processing** — handling a conversation turn
5. **Idle** — waiting for next input



Input → Parse → LLM → Execute → Respond (with memory at each step)

> 🖍️ **Exercise:** Modify the GreeterAgent to greet the user in a different language based on their preference. Add a `language` property to the instructions and test with "Hello" in Spanish, French, and Japanese.

← Back to Tutorials

4. Agent Capabilities

Memory & Context Management

Memory allows agents to persist information across conversations. MAF supports several memory providers:

Provider	Persistence	Use Case
`VolatileMemory`	In-memory only	Single-session testing
`FileMemory`	Local file system	Development & prototyping
`CosmosDBMemory`	Azure Cosmos DB	Production multi-instance

```
import { VolatileMemory } from "@microsoft/agents";

const agent = new Agent({
  name: "MemoryAgent",
  instructions: "Remember user preferences.",
  memory: new VolatileMemory({
    maxTokens: 4000,
  }),
});

// In conversation
await agent.memory.set("user_preferred_language", "Spanish");
const lang = await agent.memory.get("user_preferred_language");
```

Function Tools

Tools are external functions agents can invoke. Define them with clear descriptions so the LLM knows when to use them.

```
import { Tool } from "@microsoft/agents";

const weatherTool = new Tool({
  name: "get_weather",
  description: "Get weather for a city",
  parameters: {
    location: { type: "string", description: "City name" }
  },
  handler: async (args) => {
    const res = await fetch(`https://api.weather.com/${args.location}`);
    return res.json();
  }
});

agent.addTool(weatherTool);
```

Hands-On Lab: Memory & Tools

1. Configure `FileMemory` in your agent
2. Create a tool that saves notes to a local file
3. Create a tool that retrieves saved notes
4. Test: "Remember that I like cats" → "What do I like?"

> 💡 **Tip:** Always set clear descriptions on tools and parameters. The LLM uses descriptions to decide which tool to invoke.

> 🖍️ **Exercise:** Build an agent with persistent memory that stores user preferences (theme, language, notification preferences) and retrieves them in later conversations. Use FileMemory during development.

← Back to Tutorials

5. Tools & Extensions

Custom Tools

Tools give agents the ability to perform actions beyond text generation. Each tool has a name, description, parameter schema, and handler function.

```
import { Tool } from "@microsoft/agents";

const calculatorTool = new Tool({
  name: "calculator",
  description: "Perform basic arithmetic operations",
  parameters: {
    operation: { type: "string", enum: ["add", "subtract", "multiply", "divid
    a: { type: "number" },
    b: { type: "number" }
  },
  handler: async ({ operation, a, b }) => {
    switch (operation) {
      case "add": return a + b;
      case "subtract": return a - b;
      case "multiply": return a * b;
      case "divide": return a / b;
    }
  }
});
```

API Integration

Agents can call external REST APIs through tools.

```
const jiraTool = new Tool({
  name: "create_jira_ticket",
  description: "Create a Jira ticket",
  parameters: {
    summary: { type: "string" },
    description: { type: "string" },
    priority: { type: "string", enum: ["Low", "Medium", "High", "Critical"] }
  },
  handler: async (args) => {
    const token = await getAuthToken();
    const res = await fetch("https://your-domain.atlassian.net/rest/api/2/iss
      method: "POST",
      headers: { Authorization: `Bearer ${token}`, "Content-Type": "applicati
      body: JSON.stringify({
        fields: {
          project: { key: "PROJ" },
          summary: args.summary,
          description: args.description,
          priority: { name: args.priority }
        }
      })
    });
    return res.json();
  }
});
```

Data Connectors

MAF supports out-of-the-box connectors for common data sources:

- **Microsoft Graph** — Email, Calendar, Files, Teams
- **Azure SQL** — Database queries
- **Cosmos DB** — NoSQL document retrieval
- **SharePoint** — Document libraries

Lab: Build a Tool Suite

1. Create a weather tool (public API)
2. Create a news summarization tool
3. Create a math calculator tool

4. Combine all three in one agent
5. Test with: "What's the weather in Tokyo? Summarize AI news from today. Multiply 42 by 18."

> ✏️ **Exercise:** Build an agent that connects to a public REST API (e.g., GitHub, OpenWeather, or a mock API). The agent should be able to list resources, fetch details, and create new entries based on user commands.

← Back to Tutorials

6. Advanced Orchestration

Multi-Agent Workflows

Complex applications often require multiple specialized agents working together. The MAF orchestrator manages handoffs, context sharing, and sequencing.

```
import { Orchestrator, Agent } from "@microsoft/agents";

const triageAgent = new Agent({ name: "Triage", instructions: "Route to the r
const billingAgent = new Agent({ name: "Billing", instructions: "Handle billi
const supportAgent = new Agent({ name: "Support", instructions: "Provide tech

const orchestrator = new Orchestrator({
  agents: [triageAgent, billingAgent, supportAgent],
  defaultAgent: triageAgent,
});

// Agent can hand off
await orchestrator.handoff(triageAgent, billingAgent, { context: { userId: "1
```

Orchestration Patterns

Pattern	Description	Use Case
Router	Single entry agent delegates to specialists	Customer support
Pipeline	Agents process sequentially	Document processing pipeline
Debate	Multiple agents discuss, then synthesize	Decision support
Round Robin	Alternate between agents	Multi-perspective analysis

Context Passing

When handing off between agents, context (memory, conversation history, state) should be explicitly passed.

```
await orchestrator.handoff(fromAgent, toAgent, {
  context: {
    conversationId: convId,
    userProfile: await memory.get("user_profile"),
    sessionState: { currentTask: "troubleshooting" },
  }
});
```

Orchestration Lab

1. Create three agents: Greeter, InfoCollector, and Farewell
2. Configure a router orchestrator
3. Greeter hands off to InfoCollector after introduction
4. InfoCollector hands off to Farewell after gathering info
5. Test the full flow end-to-end

✏️ **Exercise:** Build a multi-agent pipeline for customer support: a Triage agent that identifies the issue type, then routes to Billing, Technical Support, or Account Management. Each specialist must pass conversation context and return to the Triage agent after resolution.

← Back to Tutorials

7. Production & Governance

Observability

Monitor your agents in production with Azure Monitor, Application Insights, and MAF's built-in telemetry.

```
import { TelemetryMiddleware } from "@microsoft/agents";

const agent = new Agent({
  name: "ObservableAgent",
  middlewares: [
    new TelemetryMiddleware({
      connectionString: process.env.APPLICATIONINSIGHTS_CONNECTION_STRING,
      trackConversations: true,
      trackToolCalls: true,
    })
  ]
});
```

Metrics to Track

- **Latency** — LLM response time, tool execution time
- **Token Usage** — Input/output tokens per conversation
- **Tool Success Rate** — % of tool calls that succeed
- **Conversation Length** — Turns per session
- **User Satisfaction** — Thumbs up/down, feedback

Guardrails

Guardrails enforce safety and compliance policies on agent inputs and outputs.

```
import { Guardrail, ContentSafetyPolicy } from "@microsoft/agents";

const agent = new Agent({
  name: "SafeAgent",
  guardrails: [
    new ContentSafetyPolicy({
      blockHateSpeech: true,
      blockPersonalInfo: true,
      maxInputLength: 4096,
    })
  ]
});
```

Security Best Practices

Practice	Description
Managed Identity	Use Azure Managed Identity instead of connection strings
Tool Auth	Validate authentication and authorization in every tool handler
Rate Limiting	Prevent abuse with per-user rate limits
PII Redaction	Configure guardrails to detect and redact PII
Audit Logging	Log all agent decisions and tool calls for audit

Deployment Options

- **Azure Container Apps** — Managed, serverless hosting
- **Azure Kubernetes Service** — Full orchestration control
- **Azure Functions** — Event-driven, cost-effective
- **Self-hosted** — On-premises or any Docker host

📘 **Best Practice:** Always use Azure RBAC to control which identities can create, update, invoke, or delete agents. Apply least-privilege principles.

🖍️ **Exercise:** Deploy your agent to Azure Container Apps. Configure Application Insights telemetry and verify you can see conversation traces. Set up a guardrail that blocks personally identifiable information (PII) from being shared.

← Back to Tutorials

8. Practice & Labs

Hands-On Lab Exercises

Lab 1: Basic Agent with Memory

Create an agent that remembers user preferences (favorite color, hobby, pet name) and greets the user with their details on each new conversation.

- Use FileMemory for persistence
- Store at least 3 user attributes
- Retrieve and display them on subsequent visits

Lab 2: Tool-Powered Agent

Build an agent that can perform file operations: list files, read file contents, and write to files. The agent should decide which operation to perform based on user intent.

- Three tools: `list_files`, `read_file`, `write_file`
- Proper error handling for missing files
- Test with multi-step requests like "Read notes.txt, then create a summary.txt"

Lab 3: Multi-Agent Q&A System

Build a multi-agent system where a triage agent routes questions to a technical documentation agent or a policy agent depending on the topic.

- Triage agent with routing rules
- Doc agent loads from a knowledge base
- Policy agent applies rule-based responses
- Context passed between agents

Lab 4: Production Deployment

Deploy one of your agents to Azure Container Apps with full observability:

1. Containerize your agent with Docker
2. Push to Azure Container Registry
3. Deploy to Azure Container Apps
4. Configure Application Insights
5. Test the deployed agent via REST API

Study Plan

Week	Topic	Activities
1	Foundations & Setup	Read Ch 1-2, complete setup, Lab 1
2	First Agent & Capabilities	Read Ch 3-4, build first agent, add memory
3	Tools & Extensions	Read Ch 5, build tool suite, Lab 2
4	Orchestration	Read Ch 6, multi-agent patterns, Lab 3
5	Governance & Deployment	Read Ch 7, deploy to Azure, Lab 4

> 🖍️ **Exercise:** Choose one of the labs above and complete it end-to-end. Write a short reflection (3-5 sentences) on what you learned, what was challenging, and how you would apply it to a real-world project.

← Back to Tutorials

9. References

Continue Your AI Learning Journey

- **Microsoft Agent Framework SDK Docs** — Official API reference and guides
- **Azure AI Foundry** — Model catalog, evaluation, and deployment
- **Microsoft Learn: AI Agents** — Free self-paced learning paths
- **Semantic Kernel** — Open-source AI orchestration SDK
- **AutoGen** — Multi-agent conversation framework from Microsoft

Quick Reference Guide

Task	Command / Code Snippet
Create agent	`new Agent({ name, instructions, model })`
Add memory	`agent.memory = new VolatileMemory()`
Add tool	`agent.addTool(new Tool({ name, handler }))`
Create orchestrator	`new Orchestrator({ agents, defaultAgent })`
Hand off	`orchestrator.handoff(from, to, { context })`
Add guardrail	`new ContentSafetyPolicy({ blockHateSpeech: true })`
Send message	`client.sendMessage("Hello")`

Glossary

Term	Definition
Agent	AI entity with instructions, memory, and tools
Orchestrator	Coordinates multi-agent workflows and handoffs
Tool	Function an agent can call to perform actions
Memory	Persistent storage for agent context
Guardrail	Safety/compliance policy for inputs/outputs
Handoff	Transfer conversation from one agent to another

Bonus Lecture: What's Next?

After completing this guide, you're ready to explore:

- **Copilot Extensions** — Build agents that extend Microsoft 365 Copilot
- **Custom Model Fine-tuning** — Tailor LLMs to your domain
- **Agent-to-Agent Communication** — Decentralized agent networks
- **Human-in-the-Loop** — Escalation workflows for critical decisions

Resources & Discussion

- MAF GitHub Repository: `https://github.com/microsoft/agents`
- Stack Overflow: Tag your questions with `microsoft-agents`
- Azure AI Community: Discussion forums and events

> 📘 **Tip:** Join the Microsoft Agent Framework community to share your projects, ask questions, and stay updated on the latest features.

> ✏️ **Final Challenge:** Build an end-to-end agent that solves a real problem. It should use memory, at least 3 tools, multi-agent orchestration, and be deployed with observability. Share your project with the community!